Software Architecture and Design
SOEN 343

Five Leaf Clover - HI Project Phase II

Sofia Sanchez - 40247014 - Team Leader, Design Patterns
Elizabeth Sinodinos - 40318356 - Use Case Models
Giuseppe Ippolito - 40315749 - Use Case Diagram
Michael Santella - 40264501 - System Architecture and Package Diagram
Kailee Santucci - 40296552 - Sequence Diagram

Gina Cody School of Engineering and Computer Science
Concordia University

October 20th, 2026

Table of Contents

System Architecture ……..…..2
Package Diagram ……2-3
Use Case Models ……..……. 3-8
Use Case Diagram ……..9
Sequence Diagram ……..……...10
Design Patterns ……11-13
GenAI disclosure ……13

System Architecture and Package Diagram

Description and Details of the Three Selected Layers:

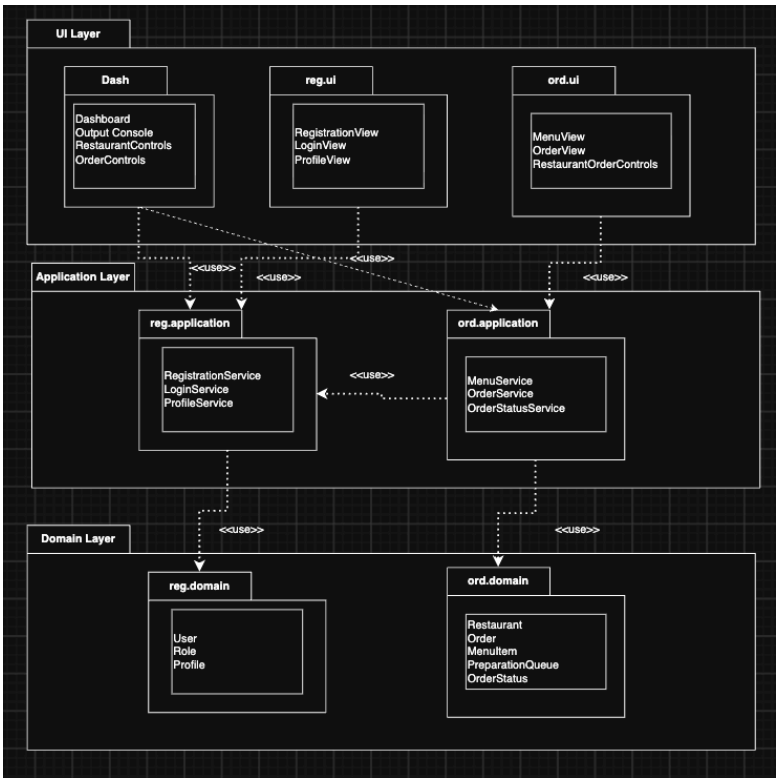
UI Layer:
This layer is responsible for what the user sees and interacts with. The UI layer contains the DASH simulator interface which includes the output console and the restaurant/order controls, the REG registration/login/profiles interfaces, and the ORD menu and order controls.

Application Layer:
This layer is responsible for coordinating application workflow and requests. The application layer contains the REG operations such as user registration and login, and ORD operations such as browsing menus, placing, accepting, and rejecting orders, as well as coordinating order status changes.

Domain Layer:
This layer is responsible for the system's business concepts and domain rules. The domain layer contains REG entities such as User, Role, Profile and ORD entities such as Restaurant, Order, MenuItem, PreparationQueue and OrderStatus. This layer also contains rules governing these entities, such as valid order-state transitions and restaurant/menu constraints.

Package Diagram

This package diagram illustrates the components and their placement within each layer chosen. The implementation will follow the same separation shown in the package diagram, with corresponding packages/folders for the UI, Application, and Domain layers.



Use Case Model (Elizabeth)

ID	UC-REG-01
Title	Create a new account
Description	The new user is able to access the system's account creation page where they can enter the required information to create a new account with a single role. The system will indicate a successful or unsuccessful account creation.
Primary Actor	New user (customer, restaurant, courier)
Preconditions	● The user is connected to a web browser with internet access ● The application is functioning ● The user has valid personal information for account creation ● The user may not have a prior account with the same account information
Postconditions	● The user will have an account created with exactly one assigned role, or the system will notify the user if account creation is unsuccessful
Inputs	● Valid personal information for account creation (email, name, password, etc) ● Role (customer, restaurant, courier)
Outputs	● A confirmation message of account creation or an error message with specifications
Main Success Scenario	1. The user requests to create a new account 2. The system requests personal information for account creation 3. The system requests a single role 4. The user provides the necessary information and submits the form 5. The system validates the user's information and selected role 6. The system creates the user's account and stores it in the database 7. The system returns a confirmation message of successful account creation

ID	UC-REG-02
Title	User Authentication
Description	The user is able to log in. The system is able to authenticate the user's credentials and create a trusted session.
Primary Actor	Existing User (customer, restaurant, courier or operator)
Preconditions	● The user has a valid account with a single role ● The system is functioning and displaying the the login page

Postconditions	● The user is successfully authenticated after submitting their account information at login ● The system validates the user's credentials and creates a trusted session ● The active trusted session allows the user to browse menus or interact with other features of the application
Inputs	● User email and password (account details)
Outputs	● User authenticated ● Trusted session created ● User able to access application features of their specific role ● If failed to authenticate, access to features of the application is denied
Main Success Scenario	1. The system displays the login page to the user 2. The user enters their valid account information and submits it to the system 3. The system authenticates the user credentials 4. The system records the user's role and creates a trusted session 5. The user is redirected to the applications homepage tailored to their role 6. The user has access to all application features for its role within the trusted session

ID	UC-REG-03
Title	Load seeded accounts
Description	The system will load seeded accounts from the configuration file for demonstration
Primary Actor	System (REG)
Preconditions	● The system is functioning ● The configuration file is accessible by the system ● The information for the seeded accounts are available in the configuration file
Postconditions	● A valid account of each role is loaded ● The seeded accounts can be utilized to login to the system
Inputs	● The configuration file with the valid account information
Outputs	● The seeded accounts of each role are loaded by the system successfully ● Confirmation or error message
Main Success Scenario	1. The system locates the configuration file 2. The system reads seeded accounts from the file 3. The system validates the seeded accounts 4. The system loads the seeded accounts 5. The system returns a confirmation message of successful loading of seeded accounts

ID	UC-ORD-01
Title	Displaying the City Map
Description	The ORD and DASH modules of the system will load a map of the initial city found in the configuration file and will display it to the user. The restaurants, their menus, the couriers and the user will be visible on the map based on their location (latitude and longitude).
Primary Actor	System (ORD and DASH modules)
Preconditions	● The system is functioning ● The configuration file is accessible by the system ● The initial map information is valid and found within the configuration file
Postconditions	● The initial city configuration is loaded into the ORD module ● A city map with icons of the different elements will be visible to the user on the dashboard of the application (DASH)
Inputs	● The configuration file with the valid information for the initial city map and the elements to be displayed on the map (icons, latitude and longitudes, status, id and name for the restaurant/menu/couriers/user)
Outputs	● All data for the elements of the map (restaurant/menu/couriers/user) are loaded into ORD ● The initial city map is loaded ● If failure to do so, an error message is displayed
Main Success Scenario	1. ORD module is functioning at application launch 2. The system locates the configuration file 3. The system reads the configuration file with the initial map and map elements (restaurant, restaurant menu items, courier information and location, etc) data 4. The system validates this data and initializes the map 5. The ORD module communicates this map initialization information to the DASH module 6. The DASH module displays the map with all elements

ID	UC-ORD-02
Title	Placing an Order
Description	The system will allow the user to place a valid order. The system will deny placing an order at a paused/closed restaurant or orders containing sold out items by disabling the users ability to do so.
Primary Actor	Customer and System (ORD module)
Preconditions	● The system is functioning ● The user is logged in to the application with a valid account and “customer” role

	● The restaurants and their menus (including item details, restaurant status, etc) are loaded and visible to the user
Postconditions	● The user is able to place an order with an open restaurant with items that are in stock ● The system will prohibit the user from ordering items that are unavailable or from closed restaurants by displaying an error message or disabling the option ● The order status is changed to “placed” when succesful
Inputs	● The user information, restaurant information, and menu (with items and their availability status) ● The quantity of each item to be ordered
Outputs	● An order with status “placed” if successful ● Confirmation or error message ● Order details and number
Main Success Scenario	1. The user identifies a valid restaurant that is open 2. The user browses the restaurants menu 3. The user adds items from the menu to their cart 4. The user places their order with the desired quantity of each item 5. The ORD module confirms the authenticity and validity of the user 6. The ORD module confirms the restaurant is open, all items are available and all quantities are valid 7. The ORD module creates the order 8. The ORD module relays this order to the restaurant’s queue 9. The ORD module transitions the order state to “placed” 10. The ORD module sends a confirmation to the user

ID	UC-ORD-03
Title	Preparing Placed Orders
Description	The ORD module will ensure that all placed orders get completed in order of arrival by placing them in the queue. No order will be refused due to want of capacity, meaning all orders will be fulfilled.
Primary Actor	Restaurant (workers), System (ORD module)
Preconditions	● A valid order has been placed to an existing and open restaurant ● The restaurant has access to its order queue ● There is at least one order in the queue to be prepared ● No order is refused due to queue capacity
Postconditions	● The order will be prepared by FIFO order ● The order status will transition to “ready” ● The couriers will be notified the order is ready for pickup
Inputs	● The user information ● The restaurant information ● The order details

	● The restaurant order queue
Outputs	● Order is successfully prepared ● Order status changed to “ready” ● A confirmation message of an order ready for pickup ● Update the preparation queue ● If more orders are in the queue, select the next order to be prepared and repeat this cycle
Main Success Scenario	1. The restaurant views and accepts newly placed orders 2. The ORD module changes the status from “placed” to “accepted” 3. The ORD module views the restaurants order queue and adds the accepted order to the queue 4. The ORD module selects the next order to be prepared in the restaurant queue based on FIFO ordering 5. The ORD module changes the order status from “accepted” to “preparing” 6. The restaurant begins preparing the order 7. The restaurant indicates the order is ready for pickup 8. The ORD module records the transition from “preparing” to “ready” 9. The ORD module repeats this process with the next order in the restaurants preparation queue

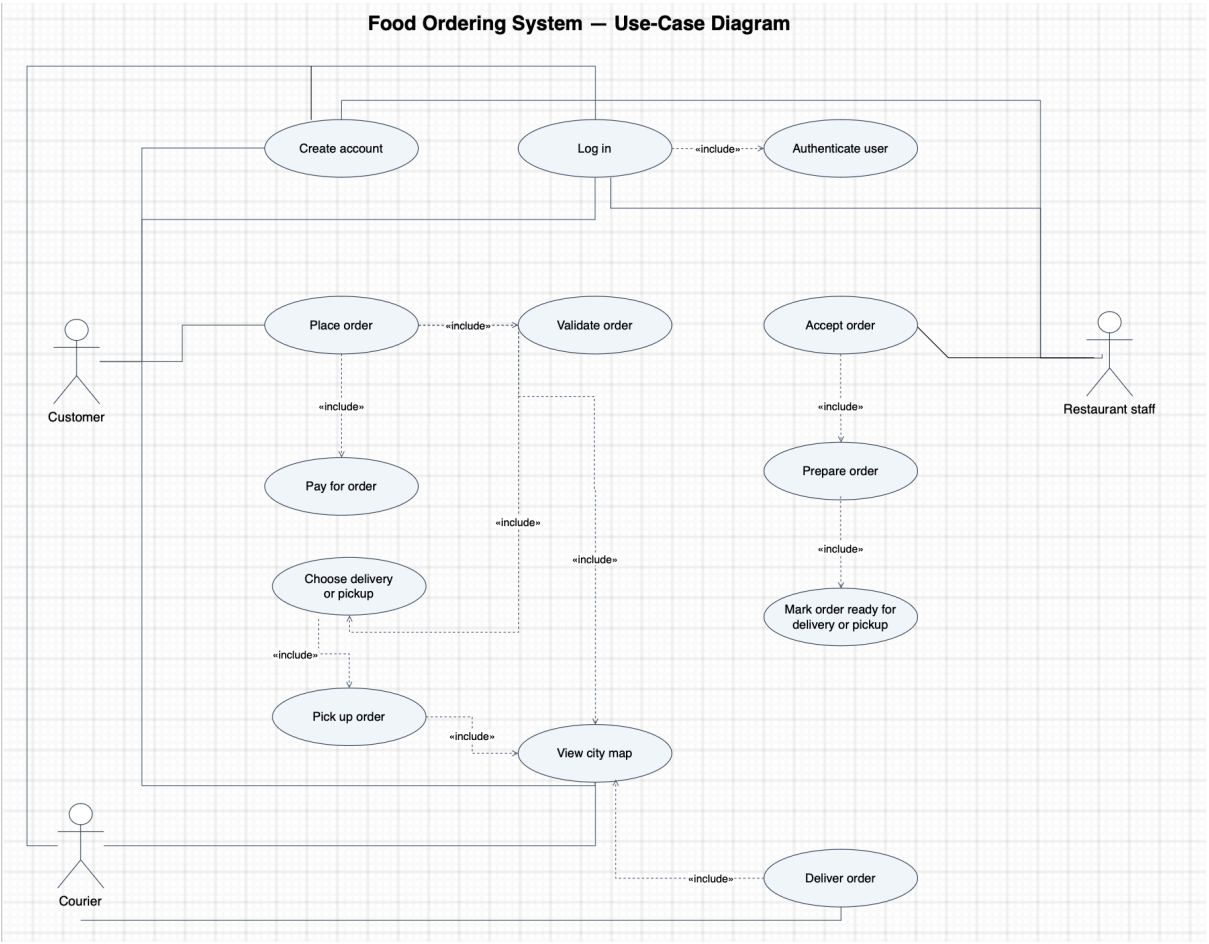
ID	UC-ORD-04
Title	Order Lifecycle Management
Description	The system’s ORD module will enforce the lifecycle management of orders, meaning that an order cannot skip a step in its fulfilment process (for example, an order can not be delivered before it is ready). It will reject any such transitions.
Primary Actor	System (ORD module)
Preconditions	● The system is functioning ● The order exists
Postconditions	● The order follows the correct transition lifecycle ● The system denies any transitions to invalid states ● The order can no longer transition once it reaches its final state (delivered/cancelled)
Inputs	● Order details ● Authenticated/valid user and user’s role ● Desired request
Outputs	● Order status updates at each transition ● The order follows the appropriate lifecycle transitions ● Any invalid transition is rejected by the system ● Confirmation or error messages are displayed to the user ● Events sent to other modules within the system as necessary
Main Success	1. A customer places and order

Scenario	2. The ORD module records the order as “placed” 3. The restaurant accepts the order 4. The ORD module records the transition from “placed” to “accepted” 5. When the order is next to be prepared in the restaurant queue, the ORD module transitions the status from “accepted” to “preparing” once the restaurant staff indicates preparation 6. The restaurant indicates the order is ready for pickup 7. The ORD module transitions the order status from “preparing” to “ready” 8. The courier picks up the order and indicates it is out for delivery 9. The ORD module transitions the order state from “ready” to “picked up” 10. The courier delivers the order to the customer and indicates the order is delivered 11. The ORD module transitions the status from “picked up” to “delivered”

ID	UC-ORD-05
Title	State Transition Event IDs and Notifications
Description	The system (ORD module) shall record every transition of the order with an event ID and send a notification to the implicated user
Primary Actor	System (ORD module), user (customer, courier, restaurant staff)
Preconditions	● An order exists ● The order transitions to a valid state
Postconditions	● The order has transitioned to the next state ● That state transition is recorded and the status of the order is updated ● The system assigns that transition with a unique event ID ● The system sends a notification to the implicated user regarding the change of status of the order (customer and restaurant staff or courier)
Inputs	● A valid order ● The current state of the order ● The next state of the order ● The time of the transition ● Role responsible for marking the transition
Outputs	● Update the order’s state ● Assign a unique event ID to the order’s transition ● Record the order transition details
Main Success Scenario	1. A valid order gets placed 2. As detailed in US-ORD-04, the ORD module associates a unique event ID to each transition of status of the order throughout its lifecycle 3. The ORD module sends a notification to the implicated users at every state transition above

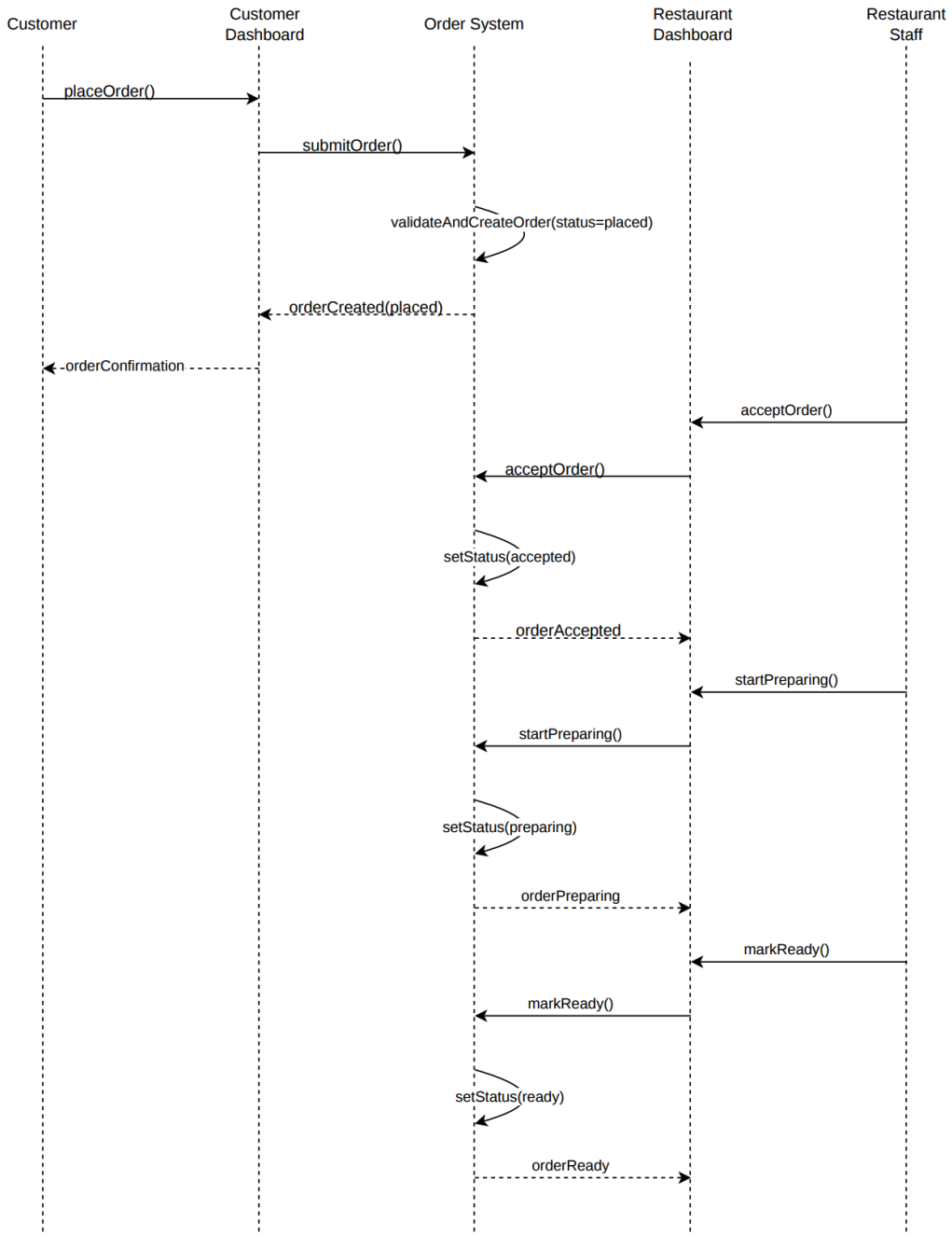
	4. The ORD module records this in the order's history along with timestamps of each transition

Use case diagram



Sequence Diagram

System Sequence Diagram: Customer Order Placement and Restaurant Preparation



Design Patterns

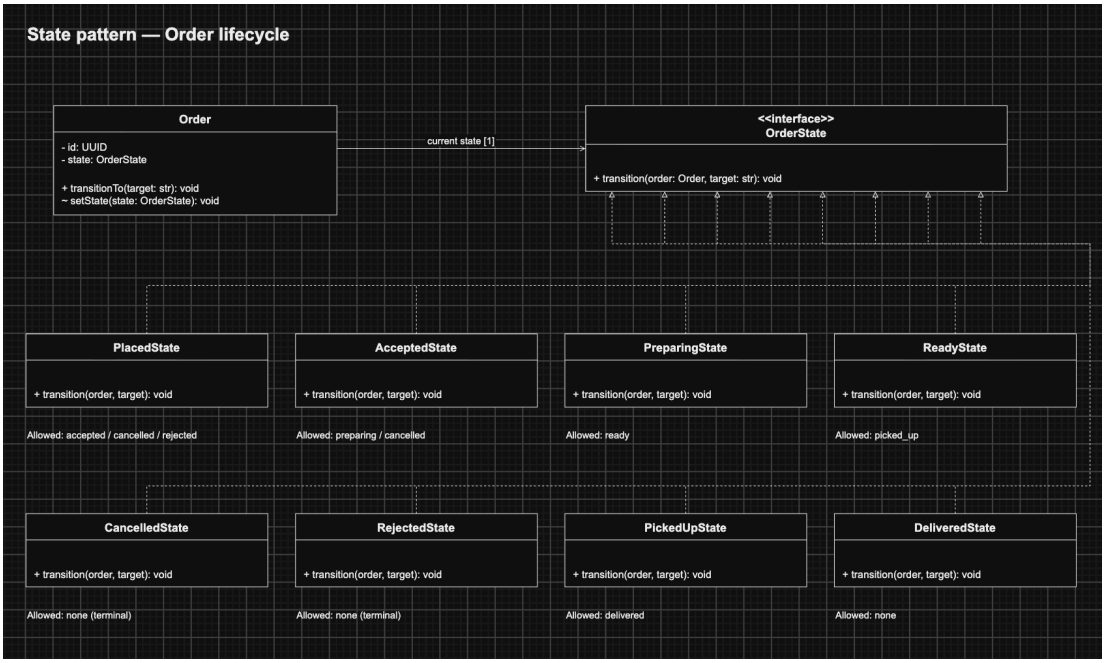
1. We chose to implement the State design pattern to manage the order lifecycle, as specified in requirement UC-ORD-04. An order has to pass through several states: placed, accepted, preparing, ready, picked up, delivered, and other optional ones such as canceled, delayed, etc. Instead of using conditional statements in the Order class to determine which operations are allowed at any given state, the State pattern lets us encapsulate the behaviour and the transition rules of each state in separate classes. This allows us to follow the single responsibility principle since the order class will delegate state specific behaviour to the corresponding state object. It also makes the system easier to maintain and extend since changes to a particular state's behaviour can be made without modifying the logic for other states. We did note that a simpler conditional-based implementation could satisfy the current requirements, but we chose the State pattern to improve the organization of our code and to be better prepared in the case of future changes.

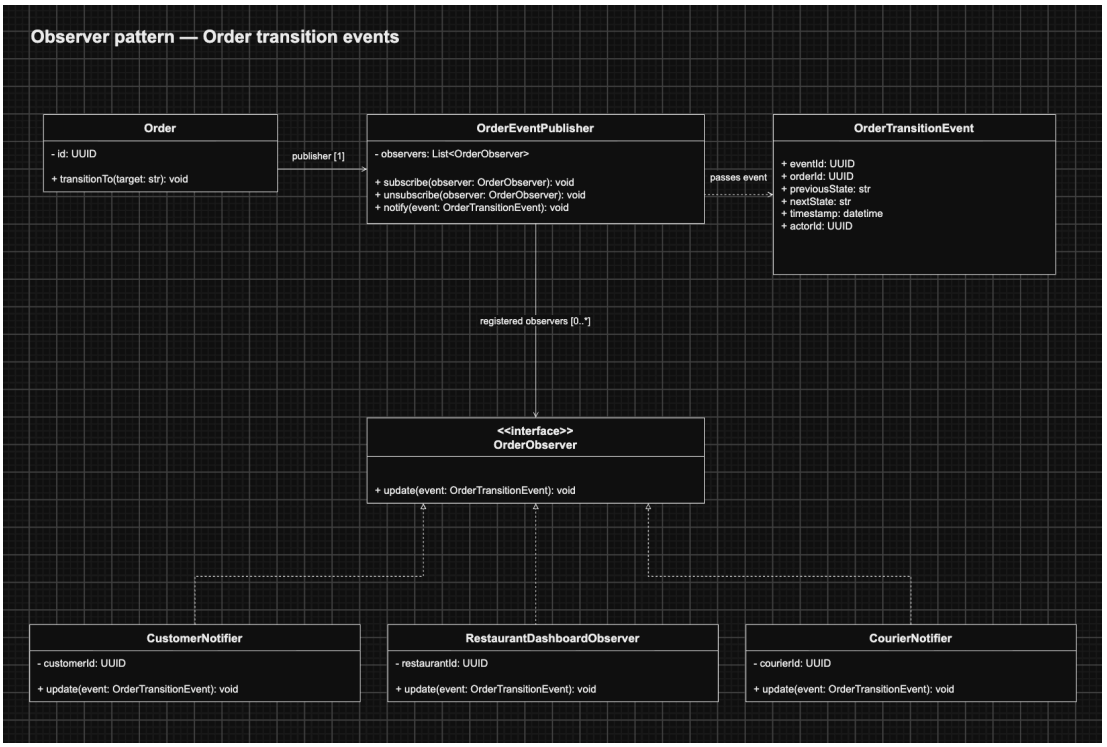
2. The Observer pattern was used to manage the notification events that take place with the state changes, as described in UC-ORD-05. When an order moves to a new state, multiple components may need to react, like notifying the customer, notifying the couriers, updating the restaurant's dashboard etc. Instead of having the order class directly call each of these components, the Observer pattern allows us to create a publisher that notifies registered observers whenever an order’s state changes. This separates the order processing from the event handling, which respects the single responsibility principle. It also respects the Open/Closed principle since new observers can be added without changing the existing order management logic if new functionality needs to be added. We considered simply calling the necessary functions directly whenever an order changes state, which would be sufficient for the current requirements. However, we chose the Observer pattern because it provides a more maintainable and flexible solution, especially as additional features requiring notifications or reactions to order events are introduced.

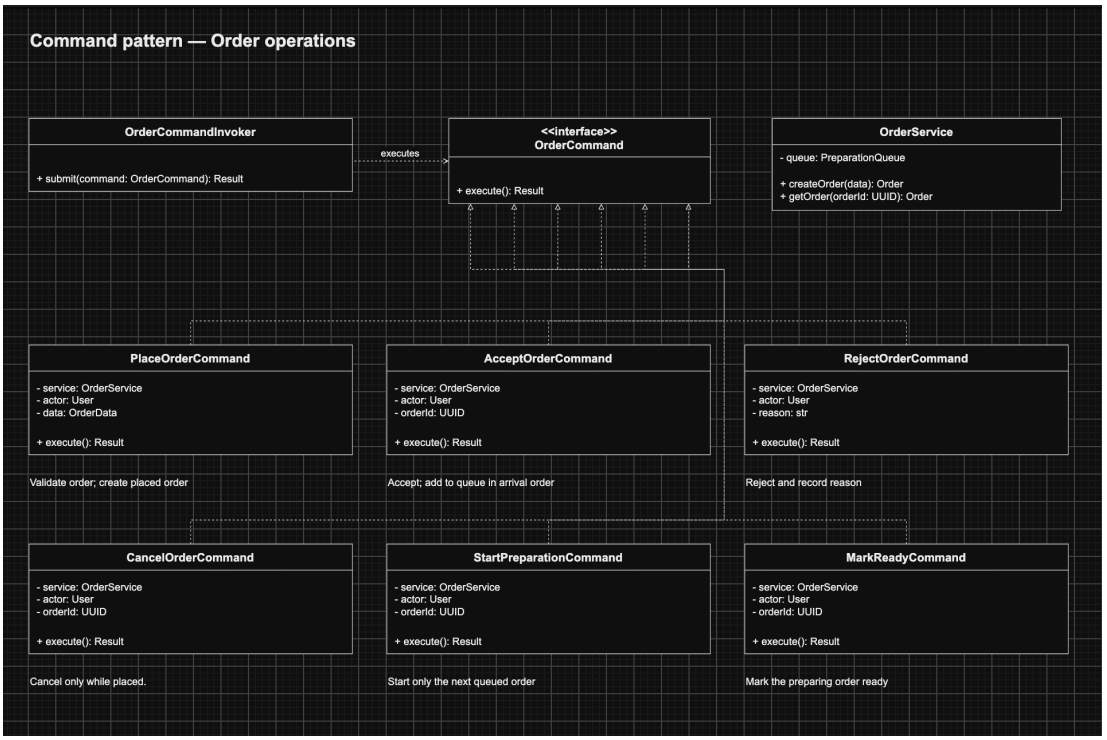
3. We chose to implement the Command design pattern to manage the different operations that can be performed on orders, as specified in use cases UC-ORD-02 and UC-ORD-03. Our application allows users to perform several operations, such as placing, accepting, rejecting, canceling, and preparing orders, each with its own validation rules and business logic. Instead of implementing all these operations directly within the OrderService class, the Command pattern allows us to encapsulate each operation in a separate command object that follows a common interface. This follows the Single Responsibility Principle, since each command class is responsible for a specific operation, and separates the code requesting an operation from the code executing it. Additionally, it makes the system easier to maintain and extend, since new operations can be introduced by creating new command classes

without significantly modifying the existing code. We considered implementing these operations as methods within a single OrderService class, which would require less steps, however, we chose the Command pattern because it provides a more organized and flexible solution as the number of operations and their associated business rules increase.

UML Class Diagrams







AI Usage Disclosure

Generative AI was utilized in the use case models section for conciseness of sentences and to brainstorm any missing components within the use case models once the use case was completed. Generative AI was used in the design pattern section to help brainstorm and compare the different design models.

Generative AI was utilized to learn how to connect different pages for the login, database management and UI styling.